SOFTWARE DESIGN DOCUMENT

Factory Traffic Management System

Event-driven traffic signal control for internal factory roads, with backend service, persistence, controller integration and monitoring dashboard

CSI Smart Tech: Backend Developer Intern Assessment V2

Document	Software Design Document (SDD)
Version	1.0
Date	28 October 2026
Status	Final for submission
Prepared by	Candidate
Prepared for	CSI Smart Tech technical review panel
Source reference	Factory Traffic Management System: Backend Developer Intern Assessment V2 (specification)

Document Control

Revision history

Version	Date	Description
1.0	28 Oct 2026	Initial release covering architecture, detailed design, interfaces, data model, verification and known limitations.

Related documents

Document	Content
README.md	Setup, design summary, demonstration steps, assumptions and requirement issues
docs/API.md, docs/postman_collection.json	API reference and request collection
docs/PROJECT_GUIDE.md	Detailed internal walkthrough of logic and event flows
docs/REQUEST_FLOW.md	File-by-file request paths for every operation
docs/SCHEDULING_LOGIC.md, docs/VEHICLE_CLEARANCE_LOGIC.md	Focused descriptions of two algorithms
docs/Backend-Files-Guide.pdf	Per-file reference of the backend
docs/Build-From-Scratch-Tutorial.pdf	Step-by-step construction guide

Table of contents

1. Introduction

1.1 Purpose · 1.2 Scope · 1.3 Definitions and acronyms · 1.4 References · 1.5 Document overview

2. System Overview

2.1 Context · 2.2 Objectives · 2.3 Functional summary · 2.4 Constraints · 2.5 Technology stack

3. Requirements Traceability

4. Architectural Design

4.1 Architectural style · 4.2 Layers and components · 4.3 Runtime view · 4.4 Deployment view · 4.5 Architectural decisions

5. Detailed Design

5.1 Junction state model · 5.2 Signal state machine · 5.3 Operating modes · 5.4 Scheduling algorithm · 5.5 Safety design · 5.6 Event processing · 5.7 Emergency handling · 5.8 Manual control · 5.9 Controller communication and failure handling · 5.10 Concurrency control · 5.11 Persistence and recovery

6. Data Design

7. Interface Design

7.1 REST API · 7.2 Status codes · 7.3 Real-time interface · 7.4 Controller port

8. User Interface Design

9. Error Handling and Logging

10. Verification and Testing

11. Installation and Operation

12. Assumptions, Requirement Issues and Decisions

13. Limitations, Risks and Future Work

14. Tool Usage Statement

Appendix A: Configuration defaults · Appendix B: Audit event catalogue · Appendix C: Source file inventory

1. Introduction

1.1 Purpose

This document describes the design of the Factory Traffic Management System, a software system that controls the traffic signals of internal road junctions inside a garment manufacturing facility. It specifies the system architecture, the detailed behaviour of the traffic-control logic, the data model, the external interfaces, the verification performed, and the assumptions and limitations of the implementation. It is intended for technical reviewers and for engineers who will maintain or extend the system.

1.2 Scope

The system comprises:

- a **backend service** (Node.js, Express, MongoDB) that receives vehicle-detection events, maintains traffic queues, runs a safe signal state machine, supports manual and emergency operation, handles device failures, keeps an audit history and survives restarts;
- a **controller integration layer** with a REST-driven simulator of the physical signal controller, designed to be replaceable by MQTT;
- a **web dashboard** (React) for monitoring, manual control and simulation.

The reference deployment controls Junction A (directions NORTH, SOUTH, EAST, WEST; phases NORTH_SOUTH and EAST_WEST). Additional junctions can be created at runtime and are controlled by the same engine. Production authentication, physical hardware integration and multi-instance deployment are outside the scope of this version (see Section 13).

1.3 Definitions and acronyms

Term	Definition
Junction	A road intersection with one signal head per approach direction.
Direction	An approach to the junction (NORTH, SOUTH, EAST, WEST).
Phase	A set of directions permitted to receive GREEN simultaneously (e.g. NORTH_SOUTH).
Conflict	Two directions belonging to different phases that are simultaneously not RED.
Stage	Position in the signal cycle: GREEN, YELLOW or ALL_RED.
Mode	Operating mode of a junction: AUTOMATIC, MANUAL, EMERGENCY or FAILURE.
Desired state	The signal state most recently commanded by the backend.
Actual state	The signal state confirmed by the physical controller; may be UNKNOWN.
Command	An instruction to the controller carrying a complete signal map and a unique command identifier.
ACK / NACK	Positive / negative acknowledgement of a command by the controller.
Effect	A side-effect instruction returned by the domain engine and executed by the application layer.
Idempotent	An operation whose repeated execution has the same effect as a single execution.
Port / Adapter	An abstract interface and a concrete implementation of it.

Term	Definition
SDD, API, REST, MQTT, ACK, TTL, UI	Software Design Document, Application Programming Interface, Representational State Transfer, Message Queuing Telemetry Transport, acknowledgement, time-to-live, user interface.

1.4 References

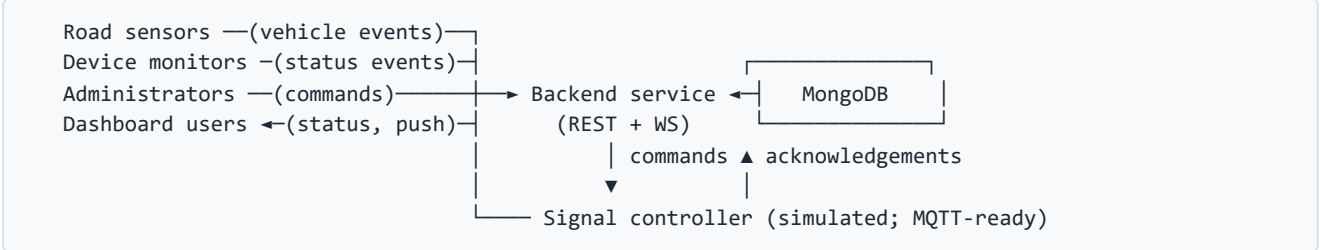
1. CSI Smart Tech, *Factory Traffic Management System: Backend Developer Intern Assessment V2* (the "specification"). Section numbers in this document written as "Spec §n" refer to it.
2. Project README and supporting documents listed under Document Control.

1.5 Document overview

Section 2 gives a system overview, and Section 3 maps each specification requirement to its implementation and verification. Sections 4 to 9 describe the architecture, detailed design, data, interfaces, user interface and error handling. Section 10 reports verification results, and Section 11 covers installation and operation. Sections 12 to 14 record assumptions, limitations and tool usage. The appendices list configuration defaults, audit event types and source files.

2. System Overview

2.1 Context



The backend is the single authority over signal sequencing. External actors submit *events* (facts) or *commands* (intents); none of them can write signal states directly.

2.2 Objectives

1. **Safety:** conflicting phases never receive GREEN simultaneously, and every phase change passes through YELLOW and ALL RED.
2. **Correctness:** queue state reflects valid sensor events; duplicates and out-of-order events do not corrupt it.
3. **Responsiveness:** emergency vehicles trigger immediate, safe preemption.
4. **Fairness:** vehicle priority and waiting time influence scheduling, and no direction waits indefinitely.
5. **Robustness:** unconfirmed or failed controller commands lead to a fail-safe state, and restarts never assume the previous physical state.
6. **Maintainability:** traffic logic is independent of HTTP, MQTT, database and UI, and is testable in isolation.

2.3 Functional summary

Capability	Description
Vehicle detection	Accepts VEHICLE_ARRIVED / VEHICLE_CLEARED events with validation, deduplication and ordering protection.
Queue management	Maintains per-direction queues of individual vehicles; counts are derived and never negative.
Automatic control	Score-based phase selection with minimum, normal and maximum green and starvation protection.
Emergency preemption	Immediate safe transition to the emergency vehicle's phase; FIFO for competing requests; timeout.
Manual control	Command-oriented green requests with expiry; return to automatic.
Failure handling	ACK timeout with retries, NACK, state mismatch, device offline, leading to FAILURE (ALL RED) with operator reset.
Audit history	38 event types recorded with timestamps, causes and command correlation.
Persistence and recovery	Full state persisted after each change; safe recovery on restart.
Multiple junctions	Junctions created via API share the engine; configuration is data.

Capability	Description
Dashboard	Live overview and detail views, manual control, simulation tools, alerts and history.

2.4 Constraints

- Expected implementation effort of 4–5 hours (Spec header); prioritisation of safety, correctness and architecture over breadth (Spec §20).
- Controller communication may be simulated through REST (Spec §13).
- Long blocking sleeps in request handlers are not permitted (Spec §6).
- A standalone MongoDB instance (no replica set) is assumed, so multi-document transactions are not available.

2.5 Technology stack

Layer	Technology
Runtime	Node.js 20+ (verified on 24), ECMAScript modules
Web framework	Express 5
Database	MongoDB with Mongoose 9
Real-time push	Socket.IO 4
Frontend	React 19, Vite, Zustand, Tailwind CSS 3, daisyUI 4, axios, lucide-react, react-hot-toast, react-router-dom
Testing	Node.js built-in test runner (`node:test` , `node:assert`)

3. Requirements Traceability

The table maps each specification area to its realisation and to the evidence of verification (test names refer to `backend/tests/`).

Spec	Requirement (summary)	Implementation	Verification
§1, §16	Separation of API, domain, persistence, controller communication, frontend; domain independent of HTTP/MQTT/DB	`domain/` (pure), `services/`, `repositories/`, `adapters/`, `controllers/`	Engine tests run with no server or database
§2	Junction A with N/S and E/W phases; extensible to more junctions; configuration not hard-coded	`junctionConfig.js`; `POST /api/junctions`	"POST junction: second junction…"
§3	RED/YELLOW/GREEN; 30 s green, 5 s yellow; ALL_RED; four safety rules	`step()`, `beginStage`, `safety.js`	"GREEN -> YELLOW -> ALL_RED -> GREEN…", "min green respected…", randomized safety test
§4	Arrival/clearance events; duplicate, delayed, out-of-order, invalid, unknown junction/type, malformed	`validators.js`, `processDeduplicated`, `applySensorEvent`	"duplicate event_id…", "validation…", "out-of-order…", "same event_id with different payload"
§5	Queues per direction; no negative values; priority, waiting time, starvation prevention	Queue as vehicle sets; `scheduler.js`	"cleared without arrival…", "truck phase beats…", "starvation…"
§6	Automatic control; state machine; consistent under concurrency; no blocking sleeps	`step()`, per-junction mutex, 500 ms ticker	"concurrent burst (spec scenario 9)…", "no unnecessary switching…"
§7	Safe emergency preemption; conflicting, repeated, stale, completed and timed-out emergencies; emergency during manual	`handleEmergencyArrival`, mode priority	Emergency test group (6 tests)
§8	Command-oriented manual control; safe transitions; return to automatic; duration and concurrency decisions	`applyCommand`, `POST …/commands`	Manual control test group (3 tests)
§9	Desired vs actual state; ACK received/delayed/missing; offline devices; reconnect; restart with pending command	`applyControllerEvent`, `handlePendingTimeout`, `applyDeviceStatus`, `recoverAfterRestart`	Controller acknowledgement group (4), device failure group (3), restart recovery group (2)
§10	Minimum backend APIs	Routes in `routes/` (Section 7)	Service tests; manual smoke test (Section 10)
§11	Audit history of listed event kinds	`AUDIT` constants, `auditLog` collection	Inspected in smoke test and dashboard

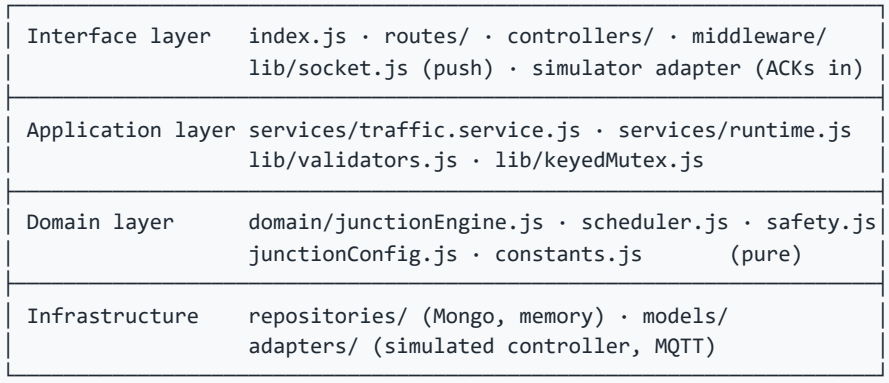
Spec	Requirement (summary)	Implementation	Verification
§12	Persistence of configuration, queues, event IDs, mode, states, commands, history; safe recovery	Four collections; `commit`; `recoverAfterRestart`	"restart: state + history survive…"; manual restart test
§13	REST simulation; MQTT-ready architecture; command IDs; timeout, retry, duplicate ACK	Controller port; simulator adapter; MQTT adapter	"ACK timeout retries same command_id…", "duplicate ACK…"
§14	Dashboard views, visual intersection, manual control, emergency and failure display, activity, simulation, refresh, error handling	`frontend/` (Section 8)	Headless browser render check
§15	Nine demonstration scenarios	Dashboard quick actions (Section 11.4)	Scenarios 3, 5, 9 automated; others by demonstration
§18, §19	Submission items, assumptions section	README, docs/	-

4. Architectural Design

4.1 Architectural style

The backend follows a **layered, ports-and-adapters** architecture with a **functional core**. All traffic decisions are made by pure functions of the form `(state, input, now) → {state, effects, result}`. Side effects (persistence, controller commands, notifications) are described as data ("effects") and executed by an application layer. This keeps the domain deterministic and testable and decouples it from infrastructure.

4.2 Layers and components



Component	Responsibility
Interface layer	HTTP routing, request/response translation, status codes, JSON error handling, real-time push.
Traffic service	Validation, per-junction serialisation, idempotency, invoking the engine, persisting state, executing effects, timer ticks, startup recovery.
Runtime wiring	Composes the service with concrete adapters (MongoDB repository, simulated controller, socket emitters): dependency injection.
Junction engine	Signal state machine, mode derivation, sensor/command/acknowledgement/device handling, recovery, status projection.
Scheduler	Phase scoring and automatic switching decisions.
Safety module	Conflict definition and safety guards.
Repository	Persistence port with MongoDB and in-memory implementations.
Controller adapter	Controller port implemented by a REST-driven simulator; MQTT implementation provided.

4.3 Runtime view

Every input follows one pipeline:

1. The interface layer receives the input (HTTP request, simulator acknowledgement or timer tick).
2. The service validates its shape (400 on failure) and resolves the junction (404 if unknown).
3. The service acquires the junction's lock, so inputs for one junction are processed strictly sequentially.

4. For sensor and device events, the event identifier is checked against processed events (duplicate → no state change).
5. The engine computes the new state, the effects and the result.
6. If the state changed, it is persisted with an optimistic version check. Persistence failure aborts the operation, and no effects are executed.
7. Effects are executed: audit records, command status updates, then command transmission through the controller port.
8. The new status is pushed to dashboards, and the response is returned.

4.4 Deployment view

Development: backend on port 3010, Vite dev server on port 5173, MongoDB on 27017. Production: a single Node.js process serves both the API and the built dashboard (`frontend/dist`) on one port. A single backend instance per deployment is assumed (Section 5.10).

4.5 Architectural decisions

#	Decision	Rationale	Consequence
1	Pure domain engine returning effects	Spec §1/§16 require domain independence and testability.	Deterministic tests with a simulated clock; side effects centralised in one place.
2	Persist state before executing effects	A command must never be issued for a state that is not durable.	A database outage blocks commands (fails safe); clients receive 500.
3	Serialised per-junction processing	Simple, deterministic consistency strategy (Spec §6).	No lost updates; junctions processed in parallel; single-instance assumption.
4	Junction-level commands carrying the full signal map	Per-direction commands can be partially applied, creating conflicts.	One acknowledgement confirms the whole junction state.
5	Stage progression gated on acknowledgement	Spec §9: an unconfirmed command must not be assumed executed.	Timers start at confirmation; a missing ACK leads to retries, then FAILURE.
6	Modes derived from facts with fixed priority	Removes ambiguity among competing control sources.	FAILURE > EMERGENCY > MANUAL > AUTOMATIC; manual resumes after an emergency.
7	Timer tick instead of sleeps	Spec §6 forbids blocking sleeps.	Timing is based on timestamps; late ticks never shorten stages.
8	Queues as sets of vehicle identifiers	Natural idempotency; no negative counts.	Robust to replays and lost-ACK retries of sensor events.
9	Configuration as data per junction	Spec §2 extensibility.	New junctions and phase layouts without code changes.
10	Socket.IO push plus polling fallback	Immediate visibility with resilience to socket loss.	Dashboard is never stale for more than a few seconds.

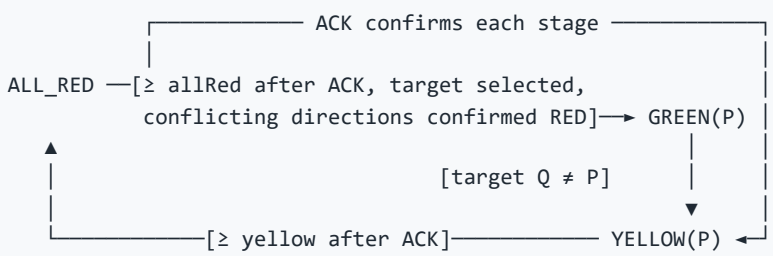
5. Detailed Design

5.1 Junction state model

Each junction is represented by a single state object, persisted as one document. Times are epoch milliseconds.

Attribute	Description
junctionId, name, config	Identity; directions, phases, timings, priorities, scoring parameters.
mode	AUTOMATIC | MANUAL | EMERGENCY | FAILURE (derived).
stage, phase, targetPhase, lastGreenPhase	Cycle position, the active phase (null during ALL_RED), the planned next phase, the last phase served.
stageStartedAt, stageConfirmedAt	Command issue time and controller confirmation time of the current stage.
desiredSignals, actualSignals	Commanded and confirmed signal maps (actual may be UNKNOWN).
controllerStatus, devices	Controller, signal-head and sensor health; last status timestamp per device.
queues	Per-direction lists of vehicles {vehicleId, vehicleType, eventId, sequenceNo, arrivedAt, sensorTime}.
lastSequence, clearedVehicles	Ordering metadata; tombstones of cleared vehicles (max 200).
phaseLastServedAt	Last service time per phase (sensor fallback).
emergency.requests, manual, failure	Active emergency requests; manual override; failure record.
pendingCommand, lastCommandResult, recentCommandIds, commandSeq	Outstanding command; last outcome; recent confirmed IDs; ID counter.
version	Optimistic concurrency counter.

5.2 Signal state machine



From	To	Guard
ALL_RED	GREEN(P)	No pending command; ALL_RED confirmed for ≥ allRedMs; target P selected; every direction outside P confirmed RED.
GREEN(P)	YELLOW(P)	No pending command; selected target Q ≠ P (emergency, manual or scheduler).
YELLOW(P)	ALL_RED	No pending command; YELLOW confirmed for ≥ yellowMs.

From	To	Guard
any	ALL_RED (fail-safe)	Mode FAILURE.
any (unconfirmed)	ALL_RED (resync)	Stage never confirmed (e.g. after a failure reset).

No transition GREEN(P) → GREEN(Q) exists. Emergency and manual requests influence only the target selection; YELLOW and ALL_RED are never skipped.

5.3 Operating modes

The mode is recomputed after every input: `failure ? FAILURE : emergencyRequests ? EMERGENCY : manual ? MANUAL : AUTOMATIC`. Each change is audited as MODE_CHANGED.

Mode	Target selection	Entry / exit
AUTOMATIC	Scheduler (5.4)	Default when no other condition holds.
MANUAL	Phase of the requested direction, held	MANUAL_GREEN_REQUEST; exits on RETURN_TO_AUTOMATIC or expiry.
EMERGENCY	Phase of the oldest active request, held	Fresh emergency arrival; exits when all requests are cleared, cancelled or timed out.
FAILURE	None: ALL RED enforced	Timeout, NACK, mismatch, conflict report, controller or signal-head fault; exits only on operator RESET_FAILURE with healthy devices.

5.4 Scheduling algorithm

Phase score. For each phase:

```
score = Σ over queued vehicles in the phase [ weight(type) + min(waitSeconds / 15, 20) ]
      + 3 × (number of directions in the phase whose sensor is not ONLINE)
weight: EMERGENCY 10, TRUCK 4, FORKLIFT 3, EMPLOYEE_VEHICLE 1
```

Waiting time is measured from server receipt of the arrival event.

Decision while GREEN (first applicable rule wins):

#	Condition	Decision	Reason code
1	No other phase has demand	Hold	NO_COMPETING_DEMAND
2	Green elapsed < 10 s	Hold	MIN_GREEN
3	A vehicle in another phase waited ≥ 90 s	Switch	STARVATION_PROTECTION
4	Current phase has no demand	Switch	CURRENT_PHASE_EMPTY
5	Green elapsed ≥ 60 s	Switch	MAX_GREEN_REACHED
6	Green elapsed ≥ 30 s and best other score ≥ current score	Switch	HIGHER_SCORE_AFTER_GREEN
7	Otherwise	Hold	KEEP_GREEN

Selection after ALL_RED: the longest-starving phase (≥ 90 s) first; otherwise the highest score, with ties resolved in favour of the planned target; with no demand, round-robin. The reason code is recorded with each TRANSITION_STARTED audit event.

5.5 Safety design

A signal map is *safe* if all non-RED directions belong to a single phase (YELLOW counts as permissive). Safety is enforced by four independent mechanisms:

1. **Construction:** signal maps are produced only by `signalsForPhase` (one phase coloured, all others RED).
2. **Assertion:** `assertSafeSignals` rejects any unsafe map before a command is issued.
3. **Confirmation guard:** GREEN is commanded only when every conflicting direction is confirmed RED; UNKNOWN does not qualify. Otherwise a SAFETY_HOLD is recorded and ALL RED is re-issued.
4. **Acknowledgement gating:** no stage advances while a command is unconfirmed.

In addition, a controller report of a conflicting state causes CONFLICT_DETECTED and FAILURE. A randomized test of 4,000 mixed inputs verifies the invariants after every step (Section 10).

5.6 Event processing

Concern	Design
Validation	Shape validation in the application layer (required fields, enumerations, identifier pattern, ISO timestamps, sequence numbers); junction-dependent validation (direction) in the engine.
Duplicate detection	`event_id` is the idempotency key, stored with a payload hash (unique index, 7-day TTL). Same ID and payload → DUPLICATE (200, no change); same ID with a different payload → 409.
Second idempotency layer	Queues hold vehicle identities: a repeated arrival is ignored, and a clearance of an unknown vehicle is ignored. This protects against replays after partial failures.
Timestamps	Server receipt time is authoritative for scheduling and waiting time. Sensor time is used to reject events more than 60 s in the future, flag events delayed more than 30 s, suppress stale emergency preemption (over 30 s), and order device status events.
Sequence numbers	Per-direction ordering hint; non-increasing values are flagged as out-of-order but evaluated on their merits.
Out-of-order clear/arrive	A clearance leaves a tombstone; a later arrival that is older than the tombstone (by sequence or sensor time) is ignored.
Vehicle at another direction	Moved between queues (VEHICLE_MOVED).

5.7 Emergency handling

- A fresh emergency arrival creates a request {vehicle, direction, phase, detectedAt, expiresAt = detectedAt + 2 min}.
- If a conflicting phase is GREEN, YELLOW is commanded immediately (minimum green is not applied); ALL RED and GREEN for the emergency phase follow. The emergency phase is held regardless of maximum green.

- Competing requests are served first-come-first-served by server detection time; requests on the same phase are served together.
- Repeated events for the same vehicle refresh the expiry and do not duplicate the request.
- Completion occurs on VEHICLE_CLEARED for the vehicle, operator CLEAR_EMERGENCY, or timeout (EMERGENCY_TIMEOUT).
- Emergency overrides manual control. The manual override is suspended, not cancelled. New manual requests are refused (409) during an emergency.
- During FAILURE, preemption is not possible: the junction remains ALL RED and a critical alert instructs manual traffic direction.

5.8 Manual control

- MANUAL_GREEN_REQUEST(direction) selects the phase containing the direction and returns the planned safe sequence.
- Duration: default 10 minutes, configurable from 10 s to 60 min per request; automatic expiry. This removes the need to detect administrator disconnection.
- Concurrent administrators: commands are serialised; the last accepted command applies; every command is audited with its operator. An optional `expected_version` enables optimistic rejection (409).
- Refused (409) during EMERGENCY or FAILURE. RETURN_TO_AUTOMATIC without an active override returns NO_OP (200).

5.9 Controller communication and failure handling

Condition	Behaviour
Command issued	Unique ID `cmd-<junction>-<sequence>`; deadline 4 s; recorded as PENDING.
ACK received	Actual state updated (missing directions = as requested); SIGNAL_CHANGED per direction; stage confirmed.
ACK delayed	Engine waits; no progression.
ACK missing	Same command ID re-sent up to 2 times, then CONTROLLER_TIMEOUT: actual UNKNOWN, FAILURE.
Duplicate ACK	Ignored (DUPLICATE_ACK).
Unknown or superseded ACK	Ignored (STALE_ACK).
NACK / FAILED	Actual UNKNOWN (or as reported); FAILURE.
Actual ≠ requested	STATE_MISMATCH; FAILURE. A reported conflict → CONFLICT_DETECTED; FAILURE.
Controller OFFLINE / UNKNOWN	Actual UNKNOWN; FAILURE.
Signal head not healthy	FAILURE.
Sensor not healthy	No failure; fixed-time fallback demand for that direction; warning alert.
Controller reconnects	Pending command re-sent immediately; FAILURE persists until reset.
In FAILURE	ALL RED re-sent every 5 s until confirmed; back-off after NACK.

Condition	Behaviour
Reset	RESET_FAILURE accepted only when the controller is not OFFLINE and all signal heads are healthy; GREEN only after ALL RED is reconfirmed.

5.10 Concurrency control

Primary mechanism: serialised per-junction processing. Each input for a junction is chained on that junction's promise queue (`keyedMutex`), so read-decide-persist-act sequences never interleave. Junctions are processed independently.

Secondary mechanism: optimistic concurrency. Each save is conditional on the state version that was read; a conflicting write fails explicitly. This detects, but does not resolve, concurrent writers in a hypothetical multi-instance deployment.

The specification's concurrent sequence (truck, emergency, manual request, duplicate emergency, ACK within 17 ms) is processed in arrival order with consistent results (Section 10).

5.11 Persistence and recovery

Persisted after every state change: configuration, mode, stage, desired and actual signals, device health, queues, sequence metadata, emergency requests, manual override, failure record, pending command and counters. History, processed event IDs and command records are written as they occur.

Not trusted after restart: physical signal state, controller status, pending command and stage timers.

Recovery procedure (executed per junction at startup):

1. Record SYSTEM_RECOVERY; mark any pending command ABANDONED.
2. Set actual signals and controller status to UNKNOWN.
3. Expire manual or emergency requests whose absolute expiry passed during downtime; recompute the mode.
4. If in FAILURE: issue the fail-safe ALL RED. If the last stage was GREEN or YELLOW: command YELLOW for that phase, then ALL RED. Otherwise: command ALL RED.
5. Resume normal sequencing only after confirmation; queues, mode, manual and emergency state continue.

6. Data Design

MongoDB is used with four collections. Times inside the junction document are epoch milliseconds; other collections use BSON dates.

6.1 junctions

One document per junction containing all attributes in Section 5.1. Unique index on `junctionId`. Complex sub-structures are stored as mixed sub-documents. Empty objects are preserved.

6.2 auditlogs

Field	Type	Description
junctionId	String	Junction (or the submitted ID for rejected events)
eventType	String	One of 38 audit types (Appendix B)
direction, vehicleId, eventId, commandId	String	Correlation fields (optional)
previousState, newState	String	Transition description (optional)
message, details	String, Mixed	Explanation and structured data (e.g. decision reason)
timestamp	Date	Server time

Index: `{junctionId: 1, timestamp: -1}`. Append-only.

6.3 processedevents

Field	Type	Description
eventId	String (unique)	Idempotency key
junctionId, kind	String	Junction; SENSOR or DEVICE
payloadHash	String	SHA-1 of the normalised payload
result, processedAt	String, Date	Original outcome; TTL index of 7 days

6.4 controllercommands

Field	Type	Description
commandId	String (unique)	Command identifier
junctionId, signals, purpose	String, Mixed, String	Target and requested signal map
attempts, lastSentAt	Number, Date	Transmission attempts
status, resolvedAt	String, Date	PENDING, ACKED, NACK, FAILED, TIMEOUT, MISMATCH, SUPERSEDED, ABANDONED

Index: `{junctionId: 1, createdAt: -1}`.

7. Interface Design

7.1 REST API

Base path `/api`; JSON request and response bodies; external field names in snake_case as in the specification.

Method	Path	Function	Success
GET	/health	Liveness check	200
GET	/junctions	List junction statuses	200
POST	/junctions	Create a junction (optional directions, phases, timings, priorities)	201
GET	/junctions/:id	Configuration and status	200
GET	/junctions/:id/status	Live status (mode, phase, stage, desired/actual signals, queues, scores, emergency, manual, failure, devices, pending command, alerts, version)	200
POST	/junctions/:id/commands	MANUAL_GREEN_REQUEST, RETURN_TO_AUTOMATIC, CLEAR_EMERGENCY, RESET_FAILURE	202 (200 NO_OP)
GET	/junctions/:id/history	Audit history (limit ≤ 500, type filter, before cursor)	200
GET	/junctions/:id/commands	Recent controller commands	200
POST	/sensor-events	Vehicle arrival / clearance	201 (200 DUPLICATE / IGNORED)
POST	/device-events	Controller, signal head or sensor status (added endpoint)	201 (200 DUPLICATE / IGNORED)
POST	/controller-events	Controller acknowledgement (ACK / NACK / FAILED)	200
GET / PUT	/simulator	Simulator settings and inbox (added endpoint)	200

Representative request (sensor event):

```
{ "event_id": "evt-10001", "junction_id": "A", "direction": "NORTH", "event_type": "VEHICLE_ARRIVED",
  "vehicle_id": "VH-501", "vehicle_type": "TRUCK", "sequence_no": 1501, "timestamp": "2026-10-
05T10:15:20Z" }
```

Representative request (command) and response:

```
POST /api/junctions/A/commands   { "command": "MANUAL_GREEN_REQUEST", "direction": "WEST", "operator":
"alice" }
202 { "status": "ACCEPTED", "plannedSequence": ["NORTH_SOUTH YELLOW", "ALL RED", "EAST_WEST GREEN"],
... }
```

7.2 Status codes and error format

Code	Usage
200	Success; idempotent replay (DUPLICATE); ignored input (IGNORED); no-op command (NO_OP)
201	Resource created; event applied
202	Command accepted; transition in progress
400	Malformed JSON; missing or invalid fields; unknown vehicle type, direction or command; future timestamp; invalid configuration
404	Unknown junction or route
409	Event ID reused with a different payload; command not permitted in the current mode; stale expected_version; duplicate junction
500	Unexpected error; no state change and no command issued

Error body: `{ "message": string, "details"?: string[] }`.

7.3 Real-time interface

Socket.IO on the backend port. Server-to-client events: `junctionUpdate` (full status after each state change) and `auditEntry` (each history record). The channel is push-only; all actions use the REST interface.

7.4 Controller port

Operation	Contract
sendCommand(command)	command = {commandId, junctionId, signals, purpose, attempt}. Completion indicates hand-over to the transport only, not execution.
onEvent(handler)	Registers the receiver for acknowledgements {command_id, junction_id, status, actual_signals?}. Acknowledgements from any adapter and from `POST /controller-events` share the same processing path.

The simulator supports auto-acknowledgement with configurable delay and behaviours NORMAL, NO_ACK, NACK and MISMATCH. The MQTT adapter maps commands to `factory/junctions/{id}/commands` and acknowledgements to `factory/junctions/+/acks` (QoS 1).

8. User Interface Design

The dashboard is a single-page React application. It displays backend state and submits commands and simulated events; it contains no sequencing logic (Spec §14.3).

View / component	Function
Overview (/)	Totals (junctions, emergencies, failures); a card per junction with mini intersection, mode, phase, controller status, queues and badges; junction creation.
Junction detail (/junctions/:id)	Visual intersection (confirmed state with desired-state hints); stage and timer; desired vs actual table with device health; alerts; queues with live waiting time and clearance; scheduler scores.
Status banners	Backend unavailable; emergency (vehicle, direction, current and next stage); failure (reason); manual override (operator, expiry, return action).
Manual control	Green request per direction with duration and operator; return to automatic; clear emergency; reset failure.
Controller panel	Pending command with attempt and timeout countdown; ACK, NACK and wrong-state acknowledgement; duplicate acknowledgement; simulator settings.
Simulation	Arrival/clearance (clearance selects from queued vehicles), duplicate re-send, per-direction emergency, random traffic, concurrent-burst scenario; device status events.
Recent activity	Audit history with category filter.

Refresh strategy: Socket.IO push for immediate updates, with polling fallback (status every 3 s, overview every 5 s, history while the socket is disconnected). A connection indicator shows Live, Polling or Backend offline.

Error handling: backend unavailability (banner, last known state retained, automatic retry); unknown junction (dedicated page); rejected operations (notification with the backend's reason); tolerance of incomplete data.

9. Error Handling and Logging

- **Validation errors** are detected before any state access and returned as 400 with field-level details. Rejected sensor events are also recorded in the audit history.
- **Domain rejections** (e.g. a manual request during an emergency) are returned by the engine as results with codes and mapped to 400/409 by the service; the attempt is audited.
- **Infrastructure errors** (database failures) abort the operation before any effect is executed and are returned as 500 without leaking internals.
- **Transport errors** when sending commands are logged; recovery relies on the acknowledgement timeout and retry mechanism.
- **Malformed JSON** returns 400; unknown routes return 404 as JSON.
- **Logging:** timestamped log lines for every audit event, non-GET or failed HTTP requests (method, path, status, duration), simulator activity, startup and recovery. Informational logs are suppressed during automated tests.
- **Startup:** failure to connect to the database terminates the process, as safe operation requires persisted state.

10. Verification and Testing

10.1 Strategy

Level	Approach
Domain unit tests	Pure engine functions exercised with a simulated clock and an auto-acknowledging helper; no I/O.
Property-style test	4,000 randomized inputs (events, ACK/NACK, wrong-state ACKs, commands, device faults, restarts) with invariants checked after every step.
Service tests	Application layer with an in-memory repository and a fake controller that asserts persistence precedes transmission.
Integration smoke test	Backend against a local MongoDB with the simulator, using HTTP requests.
UI check	Dashboard rendered and exercised in a headless Edge browser.

10.2 Automated test results

Executed on 28 October 2026 with `npm test` (Node.js test runner): **39 tests in 10 suites, 39 passed, 0 failed.**

Suite	Tests	Coverage
signal cycle	4	Initial synchronisation; stage order; minimum green and yellow duration; no progression without ACK.
controller acknowledgement	4	Retry with same command ID then FAILURE; duplicate and unknown ACK; mismatch; NACK.
emergency	6	Immediate safe preemption and hold; FIFO conflicting requests; repeated; stale; timeout; override of manual and resumption.
manual control	3	Planned sequence and hold; expiry; invalid commands and stale version.
queues and sensor events	5	Duplicate vehicle; clearance without arrival; out-of-order; arrival and clearance; unknown direction.
scheduler	4	Priority influence; no unnecessary switching; maximum green; sensor fallback.
device failure	3	Controller offline/online/reset; signal head failure; out-of-order status.
restart recovery	2	Clearance after restart; abandonment of pending command.
safety invariant (randomized)	1	No conflicting desired state; GREEN only with conflicting directions confirmed RED.
traffic service	7	Initial synchronisation; event idempotency; 409 on reuse; validation codes; concurrent burst; full restart; multiple junctions.

10.3 Integration and UI verification

Against a local MongoDB, the following were confirmed over HTTP on 28 October 2026: emergency preemption to EAST_WEST GREEN; duplicate submission returning DUPLICATE with an unchanged queue; malformed JSON returning 400; a manual request during an emergency returning 409; and, after a process

restart, persisted mode and queues with a YELLOW clearance before resumption. The dashboard rendered without console errors; triggering an emergency displayed the preemption banner and the sequence; an unknown junction displayed the not-found page.

> **Outstanding verification:** a scripted end-to-end check of every endpoint (approximately 85 cases across valid, duplicate, invalid and edge inputs) has been prepared but not yet executed. It should be run before final sign-off.

11. Installation and Operation

11.1 Prerequisites

Node.js 20 or later; MongoDB reachable at the configured URI (standalone instance sufficient).

11.2 Configuration (backend/.env)

Variable	Default	Meaning
PORT	3010	HTTP and WebSocket port
MONGODB_URI	mongodb://127.0.0.1:27017/factory-traffic	Database connection
NODE_ENV	development	production serves the built dashboard
TICK_MS	500	Engine timer interval

11.3 Procedures

```
Development:    cd backend  && npm install && npm run dev       (API on :3010)
                cd frontend && npm install && npm run dev       (dashboard on :5173)
Tests:          cd backend  && npm test
Production:     npm run build   (root)  ;  NODE_ENV=production npm start --prefix backend
Reset data:     drop the configured database and restart the backend
```

On first start with an empty database, Junction A is created and synchronised (ALL RED, then the first GREEN).

11.4 Demonstration scenarios (Spec §15)

#	Scenario	Procedure and expected outcome
1	Normal traffic	Submit mixed arrivals; phases switch per Section 5.4 with recorded reasons.
2	Priority traffic	Trucks on EAST vs employee vehicles on NORTH; EAST_WEST obtains green at normal green end.
3	Emergency preemption	Emergency on EAST while NORTH_SOUTH is green; immediate YELLOW, ALL RED, EAST_WEST GREEN held until cleared.
4	Manual override	Green request for WEST; planned sequence returned; hold; return to automatic.
5	Duplicate event	Same payload twice; second returns DUPLICATE; queue unchanged.
6	Vehicle clearance	Arrival then clearance; queue decremented; clearance of an unknown vehicle ignored.
7	Controller failure	Simulator NO_ACK or controller OFFLINE; retries, timeout, FAILURE with ALL RED; reset after recovery.
8	Restart	Restart the backend; state and history retained; recovery clearance.
9	Concurrent events	Concurrent-burst action; consistent results; no conflicting greens.

12. Assumptions, Requirement Issues and Decisions

The specification intentionally leaves several areas open (Spec §17). The decisions taken are summarised below.

#	Issue	Decision
1	Definition of conflicting movement	Directions in different phases conflict; YELLOW is permissive; turning movements and pedestrians are out of scope.
2	Per-direction command example (unsafe if partially applied)	Junction-level commands with a complete signal map; both acknowledgement forms accepted.
3	Status event with SIGNAL_CONTROLLER and a direction (ambiguous)	With a direction: signal head; without: whole controller. Separate `/device-events` endpoint added.
4	Authoritative identifier	`event_id` for idempotency; `sequence_no` as an ordering hint; vehicle-set semantics as a secondary safeguard.
5	Authoritative timestamp	Server time for decisions; sensor time for staleness, delay flags, future rejection and device ordering.
6	Delayed events	Applied and flagged; emergencies older than 30 s do not preempt.
7	Clearance without arrival	No queue change; tombstone retained.
8	Maximum waiting time	Per vehicle from server receipt; 90 s starvation threshold; 60 s maximum green.
9	Emergency versus minimum green	Minimum green waived; YELLOW and ALL RED retained.
10	Competing emergencies	FIFO; same-phase requests grouped; 2-minute timeout; operator cancellation.
11	Emergency versus manual	Emergency has priority; manual suspended and resumed.
12	Manual duration, expiry, disconnection, concurrent administrators	Default 10 min (10 s–60 min); automatic expiry; serialised last-writer-wins with audit; optional optimistic check.
13	Indefinite manual hold may starve traffic	Accepted as operator responsibility, bounded by expiry.
14	Acknowledgement timeout and retries	4 s timeout, 2 retries with the same command ID, then FAILURE (configurable).
15	Meaning of an ACK without actual state	Applied as requested; reported state takes precedence; differences cause FAILURE.
16	Desired and actual disagreement	Expected while pending; otherwise FAILURE.
17	Loss of controller communication (safety)	Immediate ALL RED fail-safe; hardware conflict monitoring is assumed necessary in a real installation.
18	Automatic recovery from failure	Not automatic; operator reset with device health checks (avoids flapping).
19	Timers after restart	Not resumed; stage restarted with safe clearance; absolute expiries continue.

#	Issue	Decision
20	Sensor failure	Degraded operation with fixed-time fallback demand.
21	Unknown vehicle types	Rejected (400) rather than assigned a guessed priority.
22	Queue drift from lost clearance events	Queues change only through events; maximum green and starvation rules bound the impact.
23	No transactions on standalone MongoDB	Ordered persistence and idempotent operations replace transactions.

13. Limitations, Risks and Future Work

Limitation / risk	Impact	Mitigation / future work
No authentication or authorisation	Any client reaching the API can issue manual commands or simulated acknowledgements.	Token-based authentication with roles; mutual TLS for controllers.
Single backend instance	No horizontal scaling or high availability.	Junction ownership via leader election or partitioned queues; the version check already detects conflicts.
MQTT adapter not integrated	Physical deployment requires integration work.	Broker deployment, adapter wiring, last-will mapped to controller OFFLINE.
Software-only safety layer	Network or host faults cannot be prevented by the backend.	Hardware conflict monitor and local fail-safe in the controller.
Immediate ALL RED on failure	Green may change to red without yellow during faults.	Configurable fail-safe sequence when the controller is still trusted.
Unbounded audit growth	Storage growth over time.	Retention policy, archiving, paginated UI.
Limited automated API and UI tests	Regressions in the HTTP and UI layers are detected manually.	Execute the prepared endpoint check; add HTTP-level and frontend tests.
Fixed two-phase visualisation	The dashboard drawing assumes four standard directions.	Generic rendering from configuration.

Further optional work: Docker Compose packaging, metrics, traffic analytics, event replay from the audit log.

14. Tool Usage Statement

AI-assisted development (Claude Code, Anthropic) was used to scaffold the backend and frontend following the candidate's established project structure, to draft the domain logic, tests and documentation, and to analyse edge cases. The candidate reviewed the implementation, executed the tests and demonstration scenarios, and is responsible for the solution. Other tools: Visual Studio Code, Node.js, MongoDB, Postman.

> Note to the author: confirm that this statement accurately reflects your own use of the tools before submission.

Appendix A: Configuration Defaults

Parameter	Default	Purpose
minGreenMs	10 000	Minimum green before a normal switch
greenMs	30 000	Normal green duration
maxGreenMs	60 000	Maximum green while others wait
yellowMs	5 000	Yellow duration
allRedMs	2 000	All-red clearance
ackTimeoutMs	4 000	Acknowledgement deadline
maxCommandRetries	2	Retries before FAILURE
failsafeRetryMs	5 000	Fail-safe re-send interval
manualDefaultMs / manualMaxMs	600 000 / 3 600 000	Manual override duration
emergencyTimeoutMs	120 000	Emergency request expiry
emergencyStaleMs	30 000	Maximum age for preemption
delayedEventMs	30 000	Delay flag threshold
futureToleranceMs	60 000	Accepted sensor clock lead
maxWaitMs	90 000	Starvation threshold
priorities	10 / 4 / 3 / 1	EMERGENCY / TRUCK / FORKLIFT / EMPLOYEE_VEHICLE
waitSecondsPerPoint / maxWaitPoints	15 / 20	Waiting-time scoring
sensorFallbackPoints	3	Demand for directions with a faulty sensor

Appendix B: Audit Event Catalogue

Category	Event types
Lifecycle	JUNCTION_CREATED, SYSTEM_RECOVERY, MODE_CHANGED
Vehicles	VEHICLE_DETECTED, VEHICLE_CLEARED, VEHICLE_MOVED, SENSOR_EVENT_DUPLICATE, SENSOR_EVENT_REJECTED, SENSOR_EVENT_IGNORED
Signals	TRANSITION_STARTED, STAGE_CHANGED, SIGNAL_REQUESTED, SIGNAL_CONFIRMED, SIGNAL_CHANGED, SAFETY_HOLD
Emergency	EMERGENCY_DETECTED, EMERGENCY_REFRESHED, EMERGENCY_STALE_IGNORED, EMERGENCY_CLEARED, EMERGENCY_TIMEOUT
Manual	MANUAL_OVERRIDE, MANUAL_EXPIRED, RETURN_TO_AUTOMATIC, COMMAND_REJECTED
Controller	CONTROLLER_ACK, DUPLICATE_ACK, STALE_ACK, COMMAND_RETRY, COMMAND_FAILED, COMMAND_ABANDONED, CONTROLLER_TIMEOUT, STATE_MISMATCH, CONFLICT_DETECTED, CONTROLLER_RECONNECTED
Devices and failure	DEVICE_STATUS_CHANGED, DEVICE_EVENT_IGNORED, FAILURE_DETECTED, FAILURE_RESET

Appendix C: Source File Inventory

Approximately 2,840 lines of backend source and tests; 1,430 lines of frontend source.

Path	Role
backend/src/index.js	Entry point, middleware, routing, startup
backend/src/domain/constants.js	Enumerations and audit types
backend/src/domain/junctionConfig.js	Defaults and configuration validation
backend/src/domain/safety.js	Conflict rules and guards
backend/src/domain/scheduler.js	Scoring and phase decisions
backend/src/domain/junctionEngine.js	State machine and input handling
backend/src/services/traffic.service.js	Application service
backend/src/services/runtime.js	Composition / dependency injection
backend/src/lib/validators.js, keyedMutex.js, utils.js, db.js, socket.js	Validation, locking, utilities, database, real-time
backend/src/middleware/error.middleware.js	Logging, JSON errors, 404
backend/src/routes/*.route.js, controllers/*.controller.js	HTTP interface (3 + 3 files)
backend/src/repositories/mongo.repository.js, memory.repository.js	Persistence port implementations
backend/src/models/*.model.js	Database schema (4 files)
backend/src/adapters/controllerPort.js, simulatedController.adapter.js, mqttController.adapter.js	Controller port and adapters
backend/tests/helpers.js, engine.test.js, service.test.js	Automated tests
frontend/src/store/useJunctionStore.js, useThemeStore.js	Client state and API access
frontend/src/pages/HomePage.jsx, JunctionPage.jsx	Views
frontend/src/components/*.jsx	UI components (12 files)